WebSockets

upgrade handshake, heartbeats and scaling

Watch the 4-min video

In one line: a WebSocket starts as an HTTP request, upgrades with `101 Switching Protocols`, and becomes one long-lived, full-duplex connection; the hard part is that every connection is state pinned to one server.

1 Why WebSockets

A chat app needs messages to appear instantly, in both directions. With plain HTTP the client must keep asking "anything new?", and the server can only answer, never start a conversation. A **WebSocket** fixes that: one long-lived connection where **either side can send at any time**. Use `wss://` (WebSocket over TLS) in production.

2 The upgrade handshake

A WebSocket starts as a normal HTTP/1.1 request that asks to switch protocols. After one exchange, the connection belongs to WebSocket:

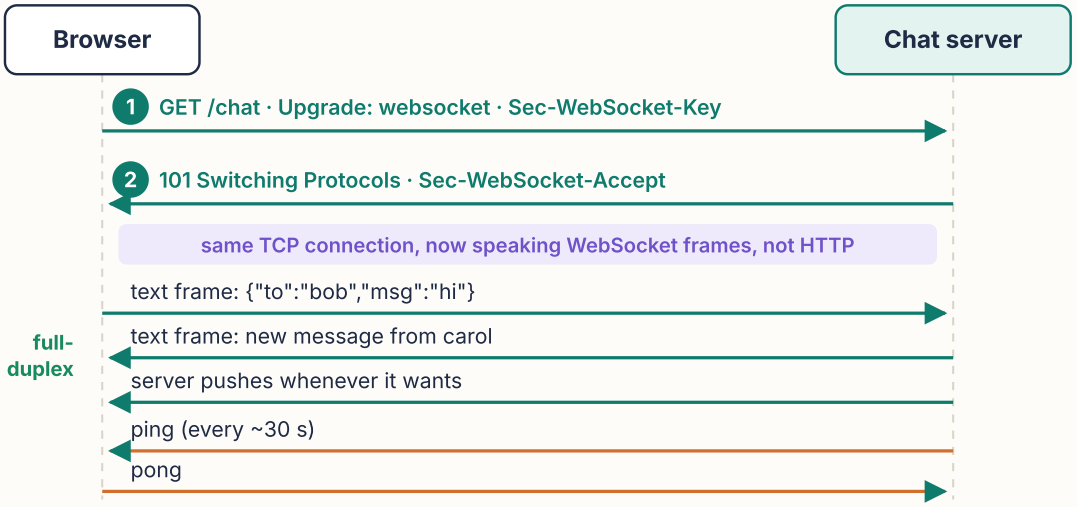


Figure 1. After the 101 response, both sides send frames whenever they like. Ping and pong are small control frames that keep the connection alive and prove the peer is still there.

On the wire, the client sends a random key and the server answers with a value derived from it:

```
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

The server proves it really speaks WebSocket by returning an **accept value derived from the key** (Base64 of SHA-1 of the key plus a fixed GUID). From then on, the **same TCP connection** carries WebSocket frames, not HTTP. Any other status (400, 401, 404, 200) means the upgrade failed and is debugged like any HTTP error.

3 Full-duplex frames and heartbeats

The connection is **full-duplex**: client and server both send whenever they want, without waiting for each other. Messages travel as **frames**, text (UTF-8) or binary, with just **2 to 14 bytes** of header each (client frames add a 4-byte mask), instead of hundreds of bytes of HTTP headers.

Connections live for hours, so you need **heartbeats**. **Ping** and **pong** control frames every ~30 seconds:

- prove the other side is still there (a phone that lost signal sends no close frame, so the socket looks open forever);
- stop load balancers and NAT gateways from closing a connection that looks idle (AWS ALB and nginx's `proxy_read_timeout` both default to 60 s);
- if pongs stop (say, two missed in a row), **close the connection and clean up**: remove it from the registry and free its memory. (Browsers answer pings automatically but JavaScript can't send them, so the server sends the pings.)

4 Example: a chat server pushing messages

Alice and Bob each hold a WebSocket to a **gateway** server, but not necessarily the same one. When Alice sends a message, her gateway stores it, looks up which gateway holds Bob's connection, and publishes it there; that gateway pushes it down Bob's socket. No polling at all.

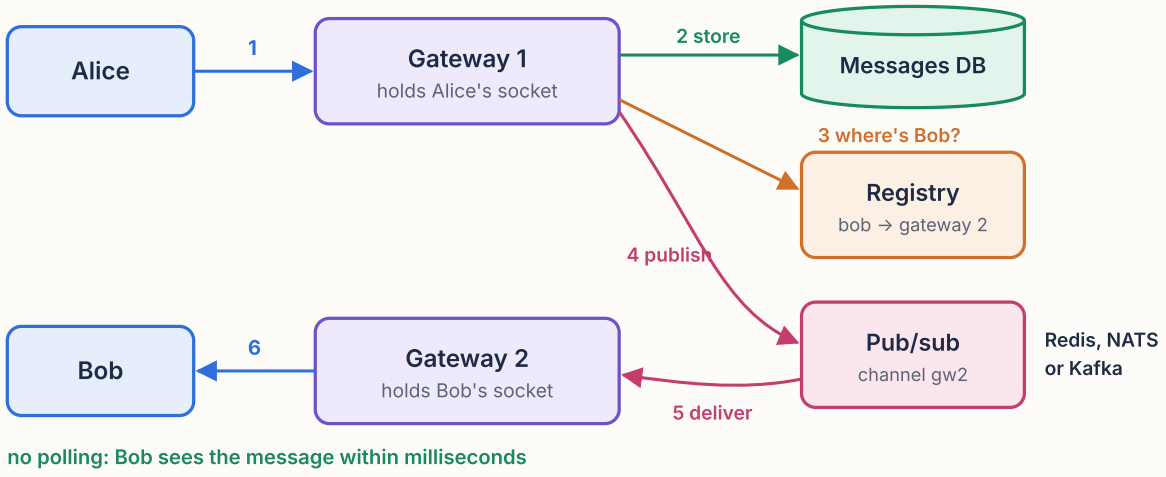


Figure 2. Gateways only hold sockets. A registry says who is connected where, and pub/sub carries messages between gateways. Storing first means Bob can still fetch the message if he is offline.

5 Scaling: every connection is state

An HTTP request lasts milliseconds; a WebSocket lasts hours, **pinned to one server**. That changes how you scale:

- **Load balancer** must support the upgrade (nginx needs `proxy_http_version 1.1` and the Upgrade/Connection headers set explicitly) and allow **long idle timeouts**, above the heartbeat interval.
- **Per-server limits** are memory and file descriptors, not CPU: each socket needs a file descriptor and some kilobytes of buffers. A tuned server (raised `ulimit -n`, async I/O) holds **hundreds of thousands** of mostly idle connections.

- **Scale out** with many gateway servers, a **registry** of who is connected where (e.g. Redis with a TTL refreshed by heartbeats), and **pub/sub** to route messages between servers.

WORKED EXAMPLE

Target: **2 million** concurrent users. At ~20 KB per idle connection, 2M × 20 KB ≈ **40 GB** of memory across the fleet. If one gateway safely holds 100,000 connections, you need 20 gateways; run **30** so that losing a server or draining one for a deploy doesn't overload the others. Heartbeats every 30 s mean 2M / 30 ≈ **67,000 pings per second** fleet-wide, tiny per server. Keep the load balancer idle timeout above 30 s, for example 120 s.

6 Plan for reconnection

When a server restarts, all its clients drop at once. If they all reconnect immediately, they **stampede** the next server (a thundering herd). So clients retry with **exponential backoff plus random jitter** (e.g. wait a random time up to 1 s, then up to 2 s, 4 s ... capped at 30 s), and **resume from the last message ID they saw** so nothing is lost. During deploys, **drain** servers gradually: stop sending new connections to a server, then close its sockets in batches with a "going away" close (code 1001), instead of killing them all at once.

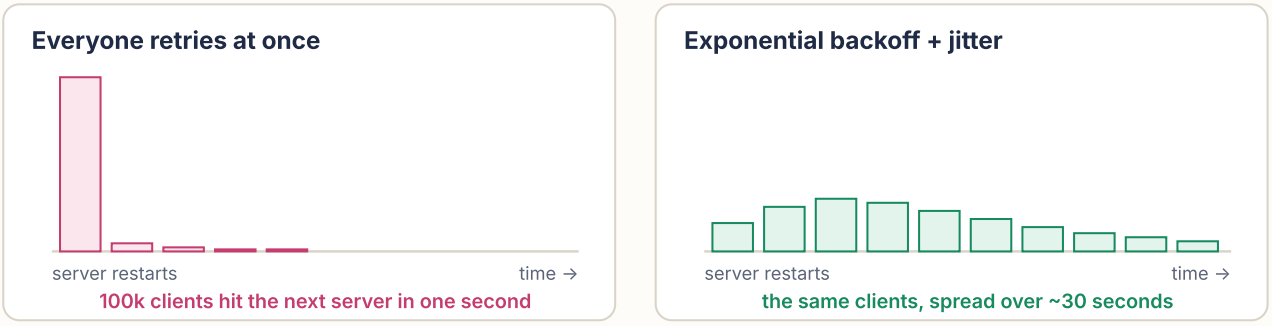


Figure 3. Left: a synchronized retry spike can knock over the server that was supposed to take the load. Right: random jitter spreads the same reconnects out.

COMMON MISTAKES

- No heartbeats: dead connections pile up, and idle-timeout proxies cut live ones at 60 s.
- Keeping the "who is connected" map only in one process's memory: messages for users on other servers vanish.
- Reconnecting instantly in a tight loop, with no backoff or jitter.
- Using WebSockets for one-way updates where server-sent events or plain HTTP would be simpler (next lesson).

7 Key terms

Upgrade: HTTP mechanism to switch protocols

Frame: unit of a WebSocket message (text, binary, control)

Connection registry: maps user → gateway

101 Switching Protocols: server agrees to switch

Ping / pong: control frames used as heartbeats

Backoff + jitter: growing, randomized retry delays

Full-duplex: both sides send at any time

Gateway: server that holds client sockets

Draining: closing connections gradually before shutdown

8 Interview questions

Q1. How does a WebSocket connection start? BASIC

As an HTTP/1.1 GET with Upgrade: websocket, Connection: Upgrade and a random Sec-WebSocket-Key. The server answers 101 Switching Protocols with a Sec-WebSocket-Accept derived from the key. Then the same TCP connection carries WebSocket frames.

Q2. Why do WebSockets need heartbeats? BASIC

To detect dead peers (a dropped phone sends no close frame) and to keep load balancers and NATs, which often cut idle connections after ~60 s, from closing healthy ones. Ping every ~30 s; close after missed pongs.

Q3. How do you scale to millions of WebSocket connections? SCENARIO

Many gateway servers (~100k connections each) behind a load balancer that supports upgrades and long idle timeouts; a registry of user → gateway; pub/sub between gateways; heartbeats; and clients that reconnect with backoff and jitter. For 2M users: about 20 gateways, plus spare capacity.

Q4. Users on gateway 1 don't receive messages sent by users on gateway 2. What's wrong? SCENARIO

Each gateway only knows its own sockets. Add pub/sub (Redis, NATS, Kafka) and a registry, so the sender's gateway can publish to the gateway that holds the recipient.

Q5. Every deploy causes a spike of errors and a CPU storm. Why, and how do you fix it? SCENARIO

Killing a server drops all its sockets and every client reconnects at once. Drain gradually (stop new connections, close in batches with code 1001), and make clients use exponential backoff with jitter and resume from their last message ID.

Q6. Connections drop exactly every 60 seconds when nobody is chatting. Why? TRICKY

A proxy or load balancer idle timeout (e.g. nginx proxy_read_timeout or ALB idle timeout, both 60 s by default). Send heartbeats more often than the timeout, e.g. every 30 s, and raise the timeout.

Q7. Can you use sticky sessions instead of pub/sub? TRICKY

Stickiness only decides where one client lands. Alice and Bob can still land on different servers, so messages still need to cross servers. Use a registry and pub/sub, or route everyone in a room to the same server by consistent hashing.

Next: SD-N-025 · SSE vs long polling vs WebSockets backendroadmap.com